Progress Report

NLP-based Scientific Article Suggestion System

Sena ÖZ
Advisor: H. Birkan YILMAZ

April 20, 2026

TABLE OF CONTENTS

1. Introduction 2
1.1. Motivation 2
1.2. Project Objective 2
1.3. Scope 2
2. System Description 2
2.1. Architecture Overview 2
2.2. Backend Pipeline 3
2.3. Data Model 7
2.4. Frontend Application 8
3. System Requirements 11
3.1. Functional Requirements 11
3.2. Non-Functional Requirements 12
4. Research Methodology and Evaluation 12
4.1. Dataset Construction 12
4.1.1. Data Collection 12
4.1.2. Preprocessing 13
4.1.3. Benchmark Dataset for Cited Paper Ranking 13
4.2. Text Representation and Similarity Methods 13
4.2.1. Field Ablation 14
4.3. Evaluation Metrics 14
4.4. Results 15
4.5. Discussion 15
5. Conclusion and Future Work 16
Bibliography 16

1. Introduction

1.1. Motivation

The volume of academic publications is growing at an unprecedented rate. Researchers at Boğaziçi University (BOUN) and similar institutions face an increasing challenge: staying up to date with newly published work from peer institutions is time-consuming and error-prone when done manually. A researcher may miss a directly relevant paper simply because it was published by an institution they do not routinely monitor.

Existing recommendation platforms such as Semantic Scholar or Google Scholar provide general-purpose suggestions, but they are not tailored to an institution's specific researcher profiles. There is a gap between the *global* stream of publications and the *local* expertise of a given research community.

1.2. Project Objective

This project aims to build an automated, NLP-based paper recommendation system targeted at BOUN researchers. The system monitors a configurable set of academic institutions via the OpenAlex API [4], detects newly published papers, embeds both the new papers and the existing BOUN researcher profiles into a shared vector space, and surfaces the most relevant matches. The core technical contribution is the evaluation and deployment of a two-stage Retrieval-Augmented Generation (RAG) pipeline [3] that combines dense embedding retrieval with local large language model (LLM) reranking.

1.3. Scope

The system covers post-2020 English-language publications from seven major research institutions (MIT, Stanford, Harvard, UC Berkeley, University of Michigan, Google, and Google DeepMind). BOUN researcher profiles are constructed from 4,608 papers authored by BOUN-affiliated researchers and fetched from OpenAlex.

2. System Description

2.1. Architecture Overview

The system is composed of three layers:

(i) **Research Layer** — Offline Jupyter notebooks for dataset construction, embedding method evaluation, and metric computation. This layer drives model selection decisions for the production system. A dedicated demo notebook (`pipeline_demo.ipynb`)

illustrates the full two-stage RAG pipeline end-to-end against live BOUN embeddings.

(ii) **Backend Layer** — A FastAPI [5] service backed by PostgreSQL 16 that runs the paper ingestion pipeline, computes embeddings, stores matches, and exposes a REST API.

(iii) **Frontend Layer** — A React 19 single-page application that allows users to follow institutions, browse recommended papers, explore researcher profiles, and manage the system via an admin panel.

The full stack is containerized with Docker Compose. The backend runs on port 8001; the frontend development server runs on port 5173. A local Ollama instance provides inference for the embedding and reranking models.

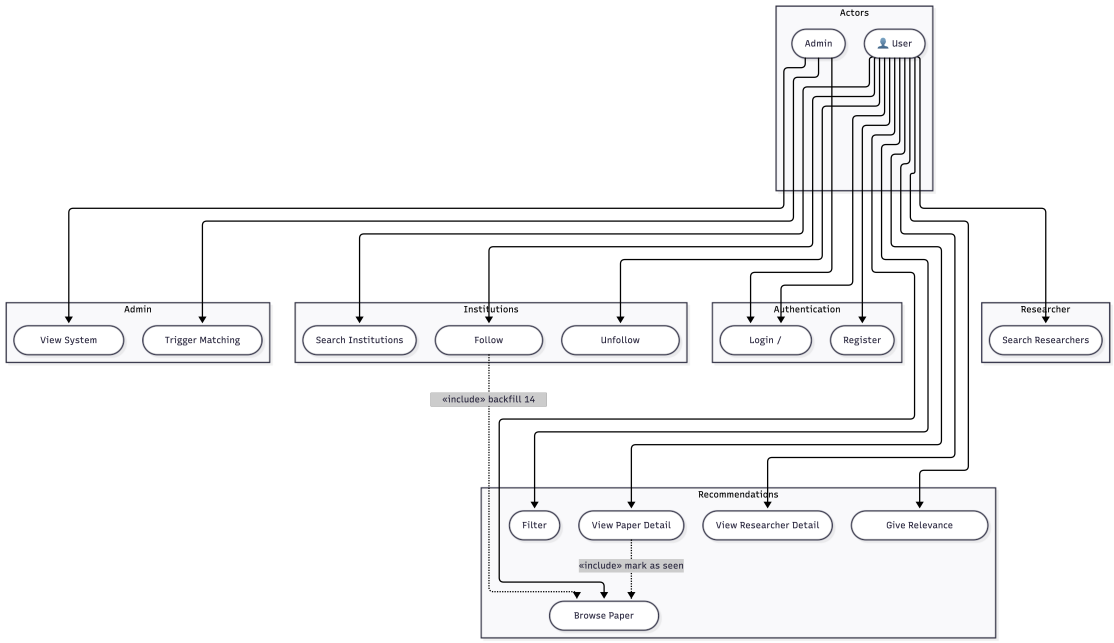


Figure 2.1. Use Case Diagram — actors (Researcher, Admin) and primary system interactions.

2.2. Backend Pipeline

On startup, the backend seeds the `researchers` table from a pre-processed CSV file (`boun.csv`) containing BOUN researcher metadata. Embeddings for each researcher's papers are computed asynchronously using Qwen3 [8] via Ollama, then stored as binary blobs in PostgreSQL.

An APScheduler job executes every six hours (`JOB_INTERVAL_HOURS = 6`). For each followed institution, the job:

(i) Fetches papers published since the last recorded cursor date via the OpenAlex API.
(ii) Encodes new paper texts (title + abstract + concept tags) with the Qwen/Nomic embedding model.
(iii) Computes cosine similarity between each new paper and all stored BOUN researcher paper embeddings (*Stage 1 — Retrieve*).
(iv) Groups hits by researcher and selects the top 20 candidates above a configurable threshold (`MATCH_THRESHOLD = 0.3`).
(v) For the top 10 candidates, invokes Llama 3.2 3B via Ollama with a structured prompt asking whether the researcher's prior work is relevant to the new paper (*Stage 2 — Rerank*).
(vi) Stores final match scores and the matched BOUN paper identifiers in the `paper_researcher_match` table.

If Ollama is unavailable, Stage 2 is bypassed and Stage 1 cosine similarity scores are used directly, ensuring graceful degradation.

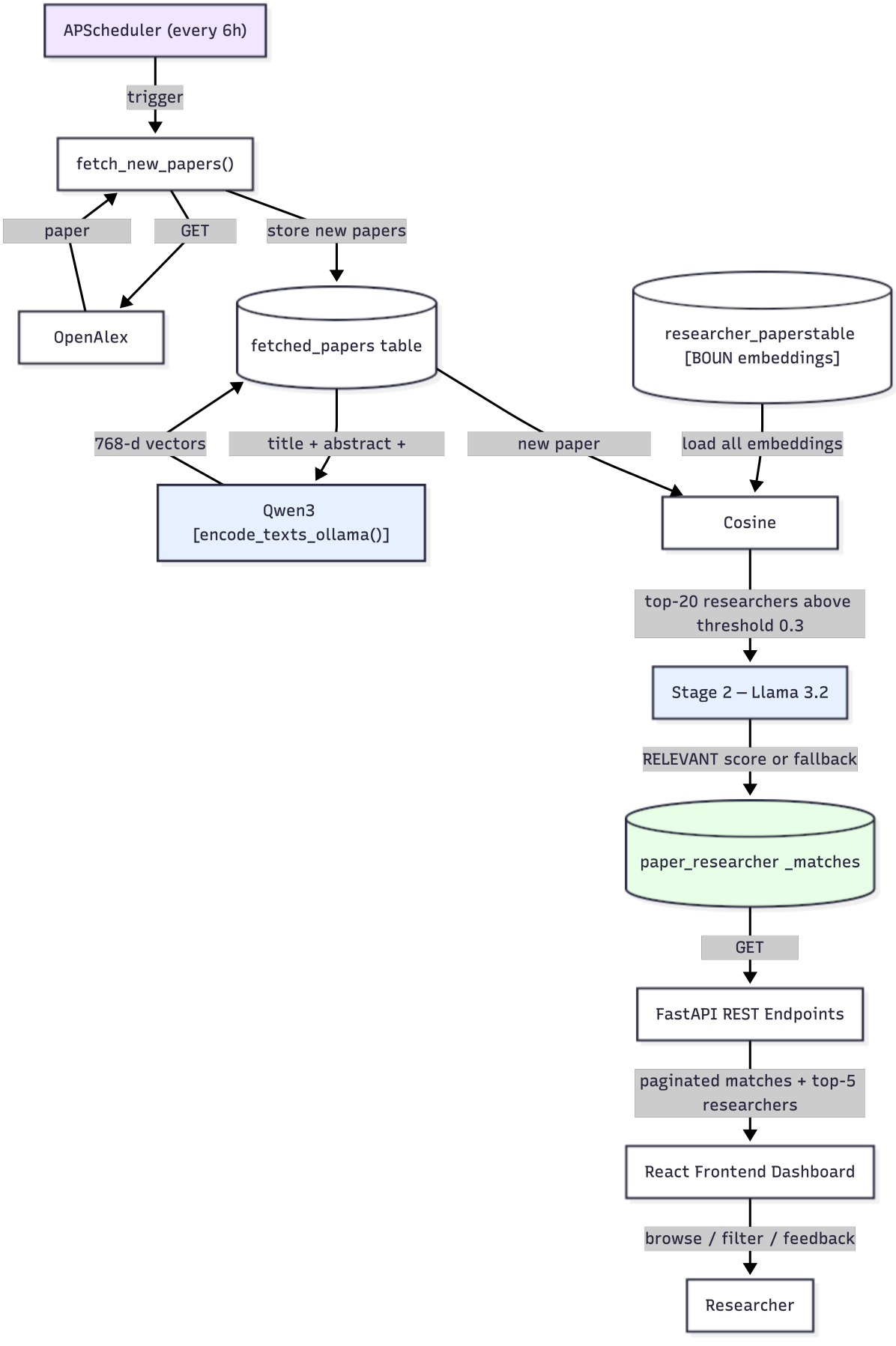


Figure 2.2. Data Flow Diagram — from OpenAlex ingestion through two-stage RAG matching to the recommendation feed.

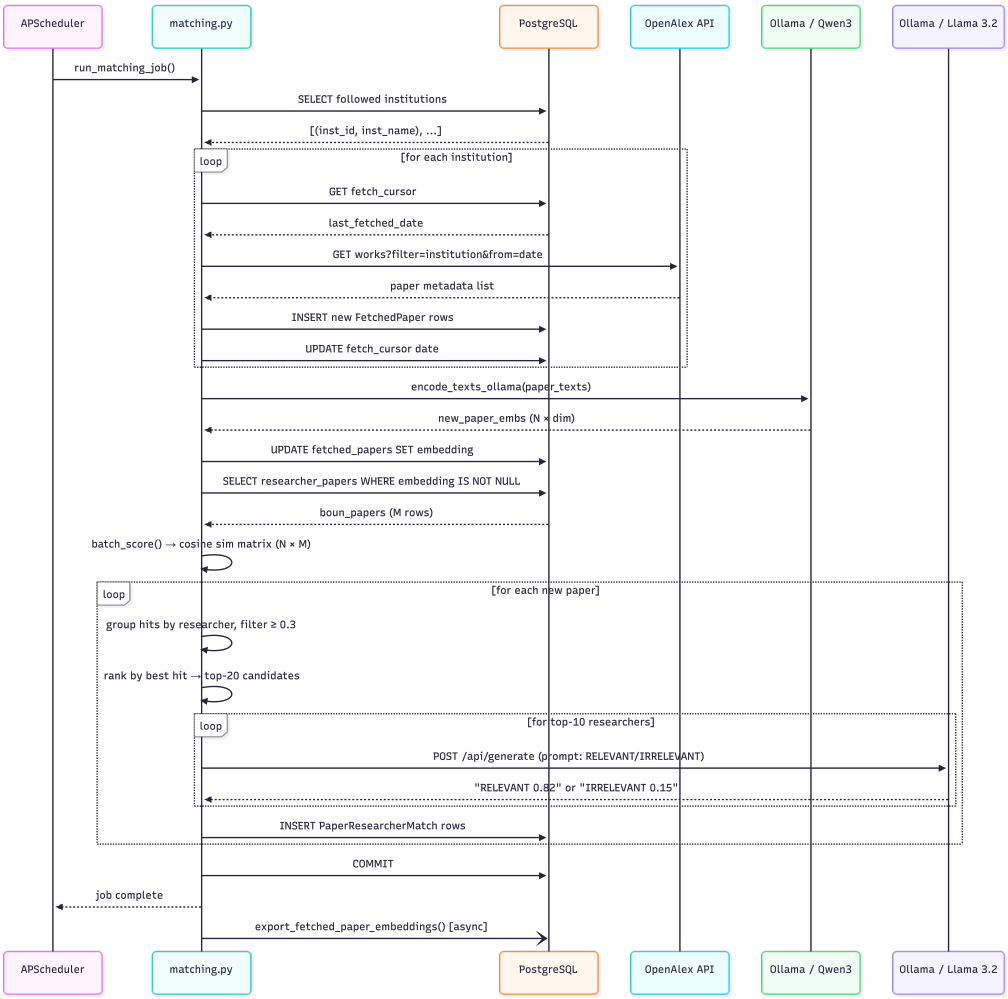


Figure 2.3. Sequence Diagram — Matching Job: Scheduler triggers paper fetch, Stage 1 embedding, Stage 2 LLM reranking, and match storage.

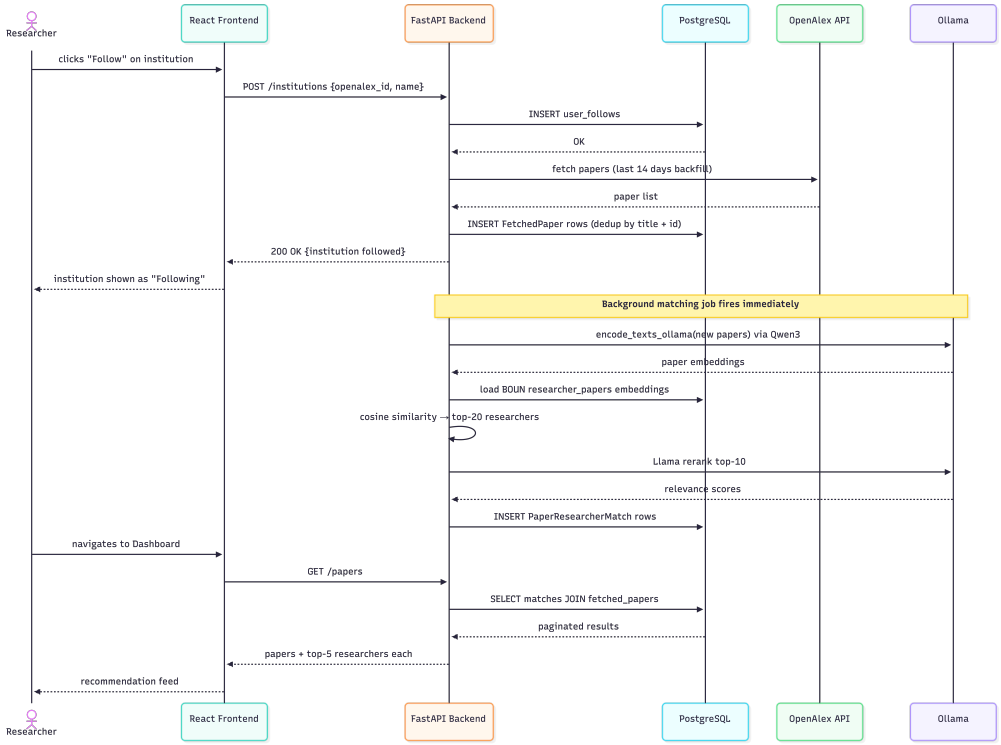


Figure 2.4. Sequence Diagram — Follow Institution: user follows an institution, triggering an immediate 14-day backfill and first-run matching.

2.3. Data Model

The key database entities are:

- `researchers` and `researcher_papers` — BOUN researcher profiles with per-paper embeddings stored as binary blobs.
- `fetched_papers` — Papers retrieved from followed institutions.
- `paper_researcher_matches` — Cosine similarity scores linking fetched papers to BOUN researchers, including matched paper identifiers (JSON) and optional LLM rerank scores.
- `user_follows` and `fetch_cursors` — Per-user institution subscriptions and incremental fetch state.
- `paper_feedback` — User relevance feedback (thumbs up/down) on paper-researcher matches, collected to support future personalization.

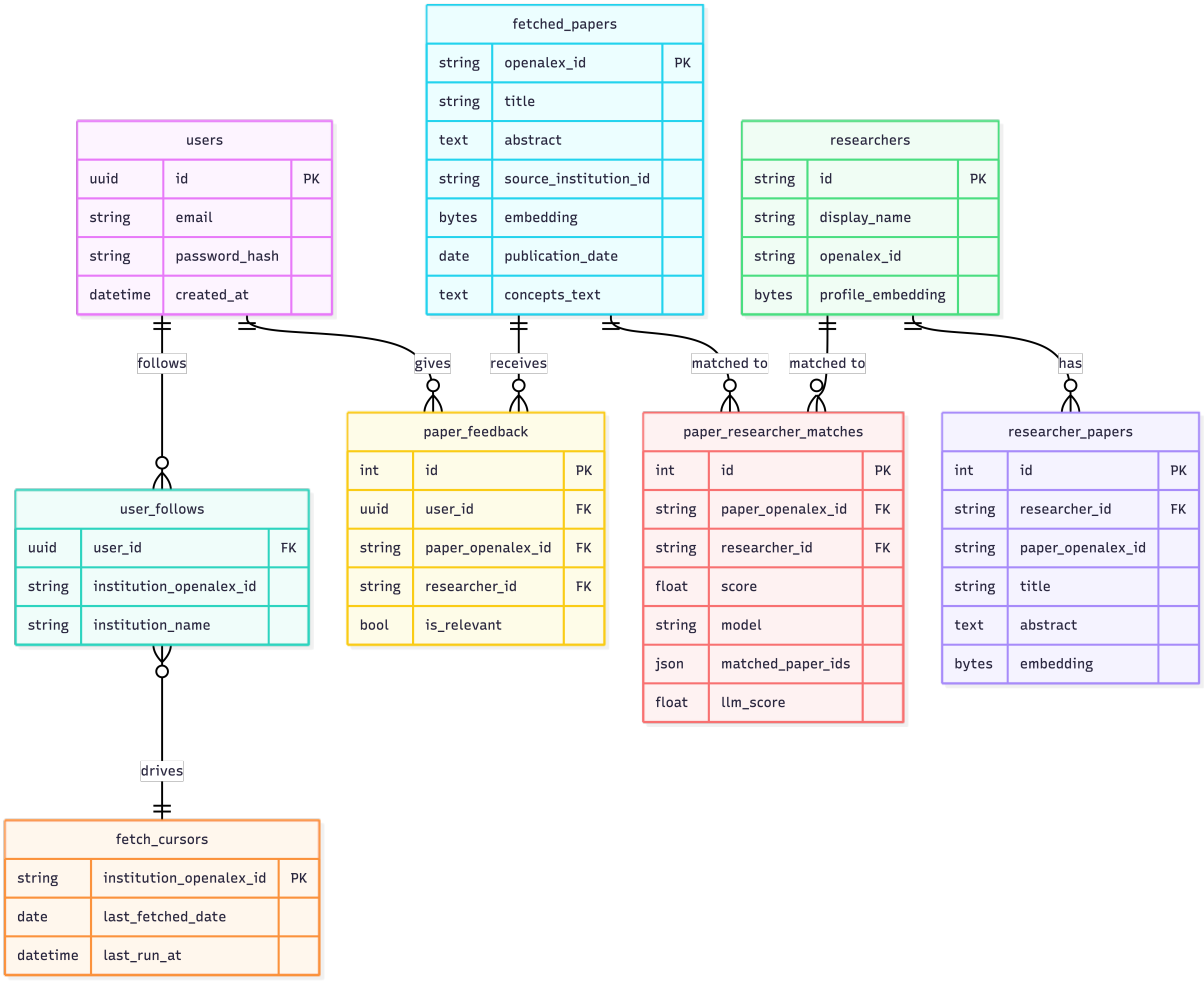


Figure 2.5. Entity–Relationship Diagram — core tables and their relationships.

2.4. Frontend Application

The frontend provides four protected views: (i) a paginated paper dashboard with top-5 matching BOUN researchers per paper and relevance feedback controls; (ii) an institution management page for following and unfollowing institutions; (iii) a researcher directory with full-text search, grouped by display name to merge duplicate OpenAlex identities; and (iv) an admin panel with system statistics and a manual job trigger. Filter controls on the dashboard allow users to restrict results by institution, minimum match score, and publication date range.

Paper and researcher detail views are rendered as in-page modals rather than separate routes, reducing navigation overhead. Each modal synchronises its state with the URL via query parameters (`?paper=<id>` and `?researcher=<name>`), making individual papers and researcher profiles directly shareable by URL. A paper is marked as *seen* when its detail modal is opened; seen papers are visually dimmed in the feed so that users can

distinguish reviewed from unreviewed recommendations.

Figures 2.6–2.7 show the main application screens.

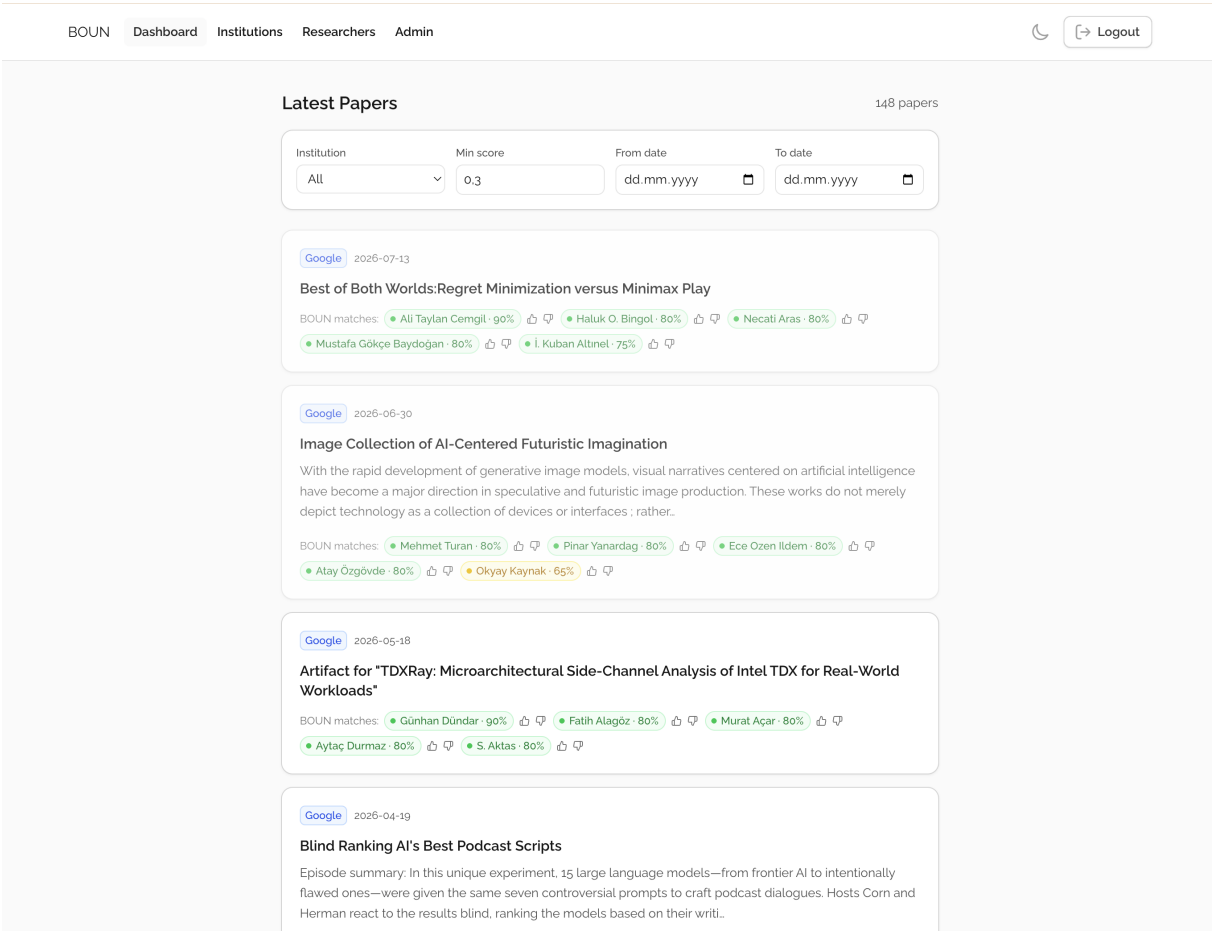


Figure 2.6. Dashboard — paginated paper feed with top-5 matching BOUN researchers, match scores, and seen/unseen state.

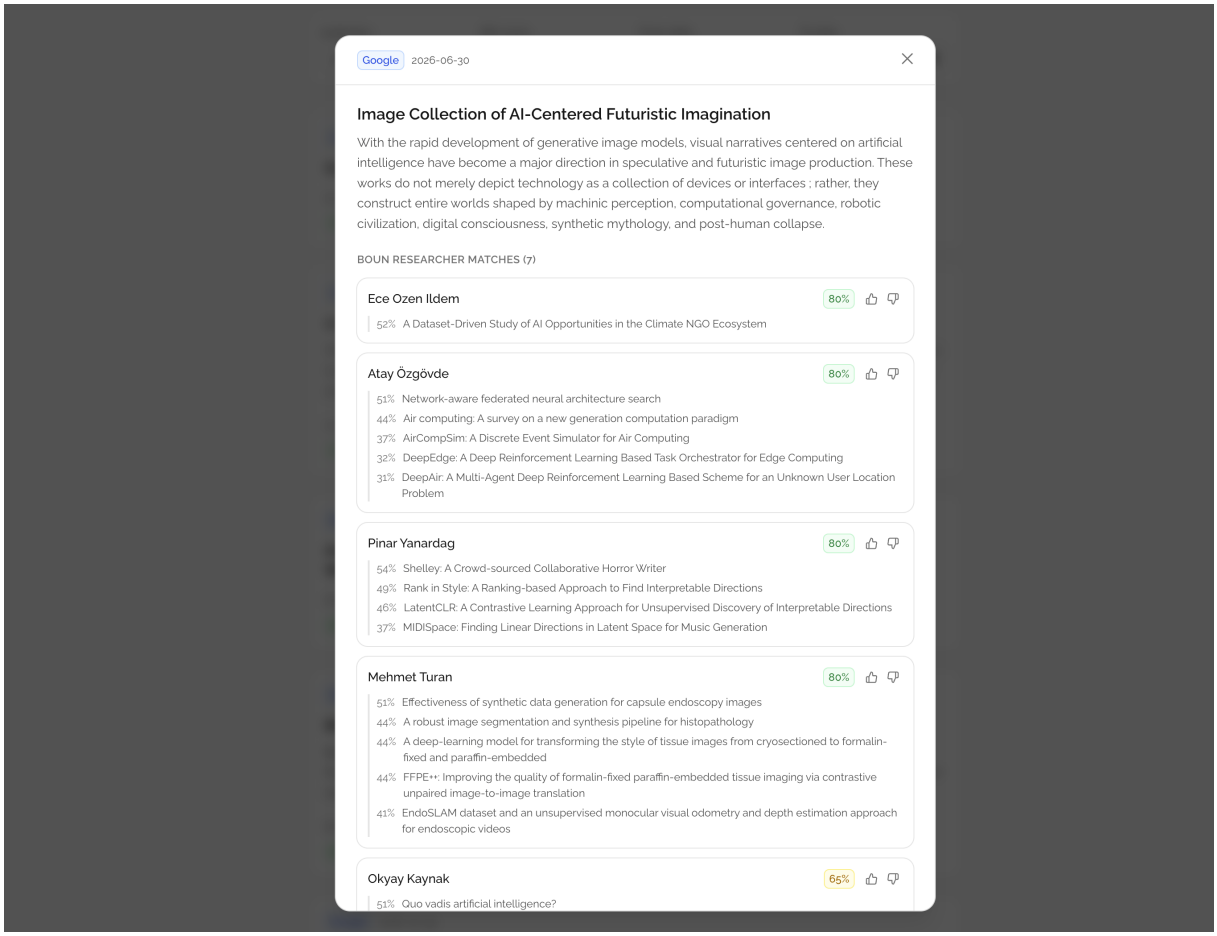


Figure 2.7. Paper Detail Modal — full abstract, concept tags, and matched researcher list with linked BOUN papers.

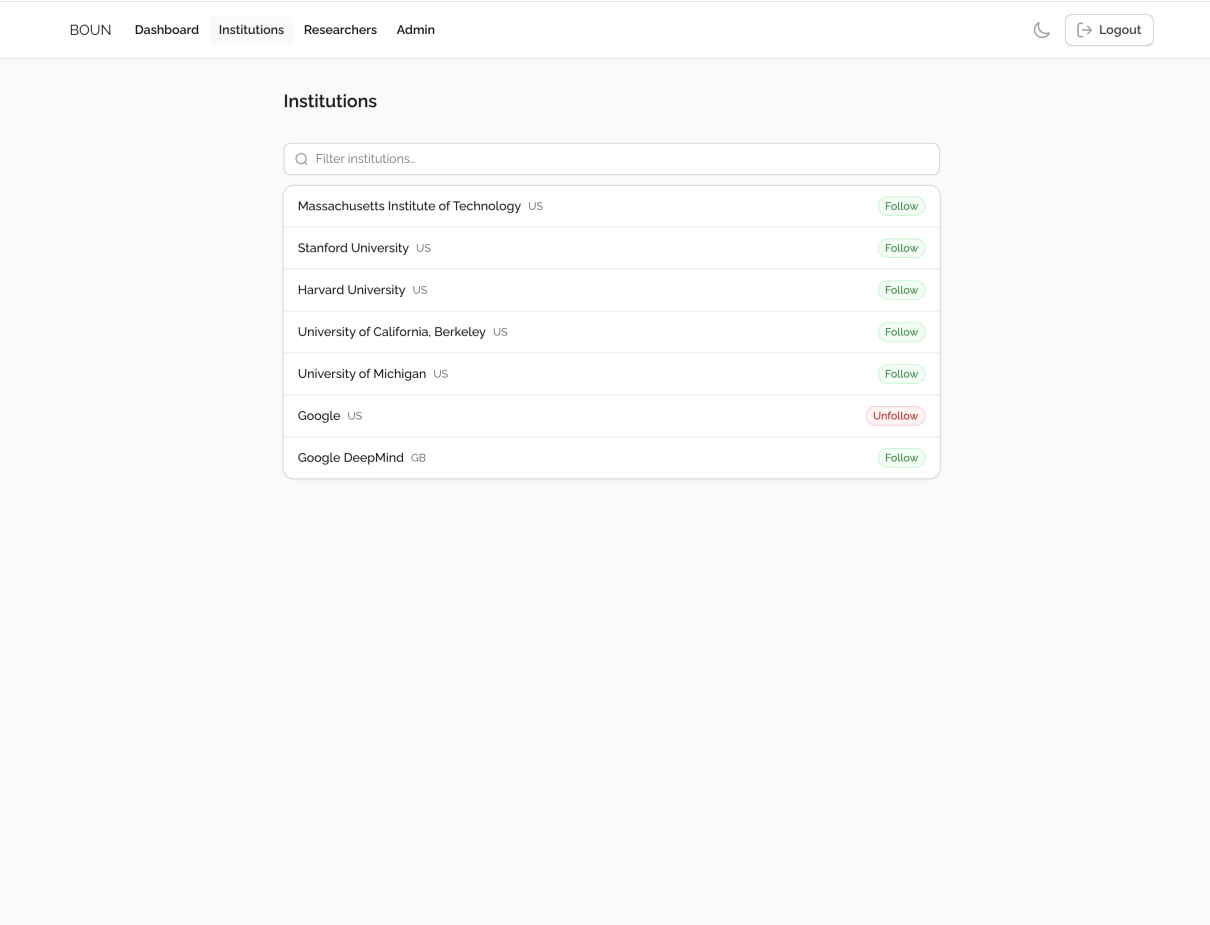


Figure 2.8. Institution Management — search, follow, and unfollow academic institutions.

3. System Requirements

3.1. Functional Requirements

(i) **Institution Following** — Authenticated users must be able to follow and unfollow academic institutions. Upon following, the system must backfill papers published within the past 14 days so that users receive an immediate set of recommendations.

(ii) **Automated Paper Ingestion** — The system must periodically fetch newly published papers from all followed institutions without manual intervention.

(iii) **Researcher-Paper Matching** — For each ingested paper, the system must compute similarity scores against all BOUN researcher profiles and store matches above a configurable threshold.

(iv) **Recommendation Display** — The user interface must display paginated recommendations showing the top-5 matching BOUN researchers per paper, along with the specific BOUN papers that contributed to the match.

(v) **Relevance Feedback** — Users must be able to mark a paper-researcher match as relevant or irrelevant; this feedback must be persisted per user.

(vi) **Filtering** — Users must be able to filter the recommendation feed by institution, publication date range, and minimum match score.

(vii) **Admin Controls** — Administrators must be able to view system statistics and manually trigger the matching job.

(viii) **Seen Tracking** — The system must record which papers a user has viewed (via modal open) and reflect this in the feed so that previously reviewed papers are visually distinguished.

(ix) **Shareable Deep Links** — Paper and researcher detail views must be accessible via stable URLs (`?paper=<id>`, `?researcher=<name>`) so that links can be shared directly.

3.2. Non-Functional Requirements

(i) **Graceful Degradation** — If the local Ollama instance is unavailable, the system must fall back to Stage 1 embedding scores rather than failing the job.

(ii) **Incremental Fetching** — The system must track the last fetch date per institution using a cursor mechanism to avoid re-fetching previously processed papers. On first follow, the cursor is initialised to 90 days prior to enable immediate backfill.

(iii) **Embedding Persistence** — All computed embeddings must be stored as serialized binary arrays in the database to avoid redundant GPU computation across restarts.

(iv) **Asynchronous Processing** — All database operations must be non-blocking; long-running embedding computations must be offloaded to thread pools.

(v) **Configurability** — Match threshold and top-K parameters must be configurable via environment variables without code changes.

4. Research Methodology and Evaluation

This chapter describes the offline research component, which drives the model selection decisions made in the production system. The research was conducted across three weekly phases.

4.1. Dataset Construction

4.1.1. Data Collection

All paper metadata was retrieved from the OpenAlex API. Two corpora were constructed:

- **BOUN Corpus**: 5,038 papers fetched for BOUN-affiliated authors; 4,608 retained after filtering for English-language papers with non-empty abstracts.
- **Followed Institutions Corpus**: 10,000 papers fetched from the seven target institutions (post-2020); 9,946 retained after the same filter.

4.1.2. Preprocessing

OpenAlex returns paper abstracts in an *inverted index* format (a mapping from tokens to their character positions). A reconstruction step converts this back to readable prose. Additional preprocessing includes HTML and URL removal, stopword filtering, and extraction of OpenAlex concept tags, which are concatenated to the title and abstract to form the final query text. The preprocessing pipeline is implemented in `research/src/preprocessing.py`.

4.1.3. Benchmark Dataset for Cited Paper Ranking

A benchmark dataset was constructed to evaluate how well each method ranks truly relevant papers above random negatives. The construction procedure (implemented in `research/cited_paper_ranking.ipynb`) is:

(i) Select papers from the followed-institution corpus that cite between 6 and 14 references also present in the corpus. This yields 174 *query papers* (P_i).
(ii) For each P_i with n cited references: collect the n references as *positives*, and sample $2n$ random papers from the corpus (excluding references and same-author works) as *negatives*, giving $3n$ total candidates.
(iii) Cache evaluation sets to `data/results_cache/week3/eval_sets.pkl`.

Query text statistics across the 174 papers: mean length 1,434 characters, median 838 characters.

For quantitative evaluation, 100 queries were randomly sampled from the 174.

4.2. Text Representation and Similarity Methods

Nine distinct methods were implemented and evaluated, covering sparse retrieval baselines, dense neural embeddings, and LLM-based reranking. All implementations are in `research/src/similarity.py`.

- **BM25** [7] — Probabilistic term-frequency ranking using the Okapi BM25 formula with saturation normalization. No GPU required.
- **TF-IDF** — Scikit-learn `TfidfVectorizer` (50,000 features) with cosine similarity.

No GPU required.

- **MiniLM** [6] — `all-MiniLM-L6-v2` sentence transformer; 384-dimensional embeddings, batch size 256.
- **SPECTER2** [1] — `allenai/specter2_base`; a transformer pretrained on citation graphs for scientific paper representation; 768-dimensional embeddings, batch size 128.
- **embeddinggemma** — Google Gemma embedding model served via Ollama.
- **Qwen** — `qwen3-embedding:latest` via Ollama; high-dimensional dense embeddings from the Qwen3 model family [8].
- **Llama reranker** — Llama 3.2 3B [2] via Ollama used as a *reranker*: given a query paper and all candidates, the model is prompted to score each candidate's relevance and return a ranked list.
- **llama+minilm** — Late fusion: min-max normalized average of Llama reranker scores and MiniLM cosine similarities.
- **qwen+minilm** — Late fusion: L2-normalized concatenation of Qwen and MiniLM embeddings, followed by cosine similarity.

4.2.1. Field Ablation

An ablation study was conducted to determine the optimal text representation. Four combinations were evaluated using MiniLM and SPECTER2 on the cross-corpus retrieval task:

- Title only: MRR 0.139–0.243
- Abstract only: MRR 0.179–0.234
- Title + Abstract: MRR 0.202–0.268
- **Title + Abstract + Concepts: MRR 0.248–0.276** (best)

Appending OpenAlex concept tags consistently improves retrieval quality. This combination is used as the default text representation in the production system.

4.3. Evaluation Metrics

Three metrics are computed for each query, implemented in `research/src/metrics.py`:

- **Hit Rate @ n%** — The fraction of the top-n ranked candidates that are true references, where n equals the number of positives for that query.
- **Mean Reciprocal Rank (MRR)** — The reciprocal of the rank position of the first true positive, averaged across queries.
- **nDCG @ n** — Normalized Discounted Cumulative Gain with binary relevance,

evaluated at position n.

4.4. Results

Table 4.1 reports mean and standard deviation of all three metrics across 100 sampled queries, ranked by mean hit rate. Top-1 accuracy (the fraction of queries where the first-ranked candidate is a true reference) is also reported.

Table 4.1. Cited paper ranking results across 9 methods (100 queries, $n = 6$–14 references per query).

Method	**Hit Rate %**	**Std**	**MRR**	**Std**	**nDCG@n**	**Top-1**
Llama reranker	**44.99**	18.87	**0.841**	0.285	**0.838**	**75%**
Qwen	37.42	14.39	0.513	0.303	0.642	25%
Qwen+MiniLM	37.22	15.99	0.526	0.331	0.640	30%
MiniLM	35.90	16.70	0.536	0.345	0.632	33%
Llama+MiniLM	34.50	15.20	0.522	0.333	0.633	30%
embeddinggemma	33.95	15.95	0.471	0.328	0.603	25%
TF-IDF	33.41	14.62	0.536	0.336	0.629	32%
SPECTER2	30.21	15.83	0.465	0.322	0.582	24%
BM25	23.09	11.70	0.410	0.304	0.528	18%

4.5. Discussion

The most striking finding is that the Llama 3.2 3B reranker substantially outperforms all dense embedding methods. With a mean hit rate of 44.99% and a top-1 accuracy of 75%, it outperforms the second-best method (Qwen embeddings) by over 7 percentage points in hit rate and 50 percentage points in top-1 accuracy. The MRR of 0.841 indicates that, on average, the first true reference appears within the top two ranked candidates.

This result is notable because Llama 3.2 3B is a relatively small model (3 billion parameters). Its advantage over dense embeddings likely stems from its ability to reason about semantic relevance across all candidates in a single generation pass, rather than independently encoding each text into a fixed-dimensional vector.

A second surprising finding is that SPECTER2 — a model specifically pretrained on the citation graph of scientific papers — underperforms the general-purpose MiniLM model and even TF-IDF on this task. We hypothesize that the cited paper ranking task, which requires fine-grained lexical and conceptual matching within a narrow candidate set, does not benefit from SPECTER2's global citation-structure pretraining objective.

Combining embeddings (Qwen+MiniLM, Llama+MiniLM) provides marginal gains

over MiniLM alone but does not close the gap with the Llama reranker.

These results informed the production system's design: Stage 1 uses Qwen dense embeddings for efficient candidate retrieval from the full BOUN researcher corpus, and Stage 2 uses Llama 3.2 3B to rerank the top-10 candidate researchers via natural language reasoning.

5. Conclusion and Future Work

This report presents the design and current state of an NLP-based academic paper recommendation system for BOUN researchers. The system is operational end-to-end: users can follow institutions, receive paginated paper recommendations matched to BOUN researcher profiles, explore researcher detail pages, and provide relevance feedback. The backend runs a two-stage RAG pipeline every six hours, combining Qwen embedding retrieval with Llama 3.2 3B reranking.

The offline research evaluation confirms that a local LLM reranker significantly outperforms dense and sparse embedding baselines on the cited paper ranking task, motivating the two-stage production architecture. Nine methods were compared across 100 evaluation queries using hit rate, MRR, and nDCG.

Planned next steps include:

- Containerize the Ollama service within Docker Compose to eliminate the host-machine dependency and enable a fully self-contained deployment.
- Publish the stack to a cloud provider.
- Replace the hardcoded JWT secret with an environment variable and rotate credentials prior to any public-facing release.
- Add container health checks and restart policies so the stack recovers automatically from transient failures.
- Complete the two-stage `qwen→llama` pipeline evaluation currently in progress and incorporate results into the method comparison table.
- Extend the benchmark to cross-corpus retrieval using the full 174-query dataset.
- Surface user relevance feedback (already persisted in `paper_feedback`) to personalize match thresholds per researcher.
- Integrate email notifications to alert researchers when high-scoring matches are found.

Bibliography

[1] Arman Cohan et al. “SPECTER: Document-level Representation Learning using Citation-informed Transformers”. In: *CoRR* abs/2004.07180 (2020). arXiv: 2004.07180. URL: https://arxiv.org/abs/2004.07180.

[2] Aaron Grattafiori et al. *The Llama 3 Herd of Models*. 2024. arXiv: 2407.21783 [cs.AI]. URL: https://arxiv.org/abs/2407.21783.

[3] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *CoRR* abs/2005.11401 (2020). arXiv: 2005.11401. URL: https://arxiv.org/abs/2005.11401.

[4] Jason Priem, Heather Piwowar, and Richard Orr. *OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts*. 2022. arXiv: 2205.01833 [cs.DL]. URL: https://arxiv.org/abs/2205.01833.

[5] Sebastián Ramírez. *FastAPI*. 2018. URL: https://fastapi.tiangolo.com.

[6] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019, pp. 3982–3992.

[7] Stephen Robertson and Hugo Zaragoza. “The probabilistic relevance framework: BM25 and beyond”. In: *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389.

[8] An Yang et al. *Qwen3 Technical Report*. 2025. arXiv: 2505.09388 [cs.CL]. URL: https://arxiv.org/abs/2505.09388.